Module 5: Assembly Stacks — Comprehensive Practice Problem Sets & Analysis

Stacks are fundamental data structures in assembly language programming, operating on a Last-In, First-Out (LIFO) principle. Utilizing hardware registers such as SS (Stack Segment) and ESP/SP (Stack Pointer), stacks facilitate procedure calls, parameter passing, register preservation, and local variable storage. This assignment features five structured programming problems with six rigorous test cases each, followed by open-ended analytical questions designed to test your mastery of low-level stack mechanics.

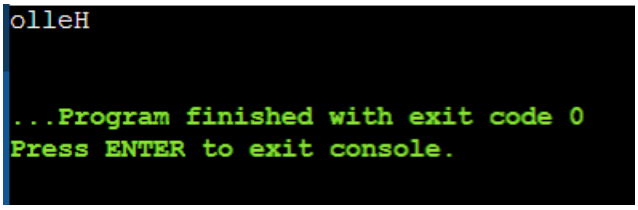
Part I: Programming Problem Statements & Test Cases (6 Test Cases per Problem)

Problem 1: Reverse a Null-Terminated String Using Stack LIFO Properties

Description: Write an x86/NASM assembly program that takes a null-terminated string stored in the .data section, pushes each character onto the stack iteratively (handling bytes as words/doublewords or padded pushes), and pops them back into memory to reverse the string. Output the reversed string using sys_write.

Test Case	Input String	Expected Output (Reversed)
Test Case 1.1	HELLO	OLLEH
Test Case 1.2	ASSEMBLY	YLBMESSA
Test Case 1.3	STACKS	SKCATS
Test Case 1.4	X86NASM	MSAN88X
Test Case 1.5	DATA	ATAD
Test Case 1.6	COMPUTER	RETUPMOC

Test Case 1.1



Test Case 1.2

```
YLBMESSA


...Program finished with exit code 0
Press ENTER to exit console.
```

Test Case 1.3

```
SKCATS


...Program finished with exit code 0
Press ENTER to exit console.
```

Test Case 1.4

```
MSAN68X


...Program finished with exit code 0
Press ENTER to exit console.
```

Test Case 1.5

```
ATAD


...Program finished with exit code 0
Press ENTER to exit console.
```

Test Case 1.6

```
RETUPMOC

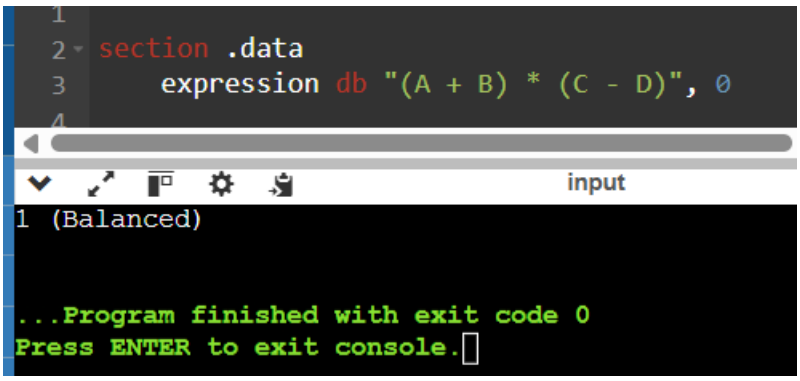

...Program finished with exit code 0
Press ENTER to exit console.
```

Problem 2: Stack-Based Balanced Brackets Validator

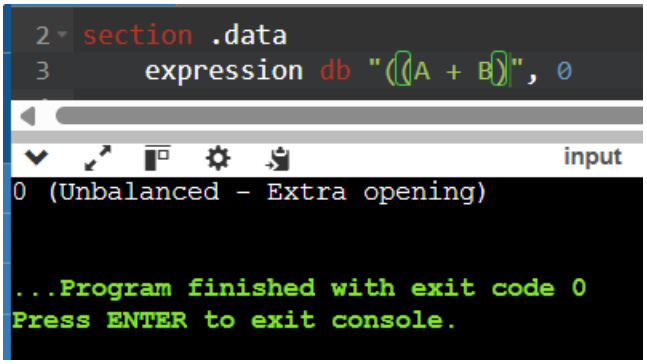
Description: Implement an assembly subroutine that analyzes an arithmetic expression containing parentheses '(' and ')'. Push opening brackets onto the stack and pop on closing brackets. Return 1 in EAX if balanced, 0 if unbalanced.

Test Case	Input Expression	Expected Result (EAX Value)
Test Case 2.1	(A + B) * (C - D)	1 (Balanced)
Test Case 2.2	((A + B)	0 (Unbalanced - Extra opening)
Test Case 2.3	A + B) * (C	0 (Unbalanced - Unexpected closing)
Test Case 2.4	((X))	1 (Balanced)
Test Case 2.5	)A + B(	0 (Unbalanced - Wrong order)
Test Case 2.6	A + (B * (C - D)	0 (Unbalanced - Unclosed)

Test Case 2.1



Test Case 2.2



Test Case 2.3

```
section .data
    expression db "A + B) * (C", 0
```

```
input
0 (Unbalanced - Unexpected closing)


...Program finished with exit code 0
Press ENTER to exit console.
```

Test Case 2.4

```
section .data
    expression db "((X))", 0
```

```
inpu
1 (Balanced)


...Program finished with exit code 0
Press ENTER to exit console.
```

Test Case 2.5

```
section .data
    expression db ")A + B(", 0
```

```
input
0 (Unbalanced - Wrong order)


...Program finished with exit code 0
Press ENTER to exit console.
```

Test Case 2.6

```
section .data
    expression db "A + (B * (C - D)", 0
```

```
input
0 (Unbalanced - Unclosed)

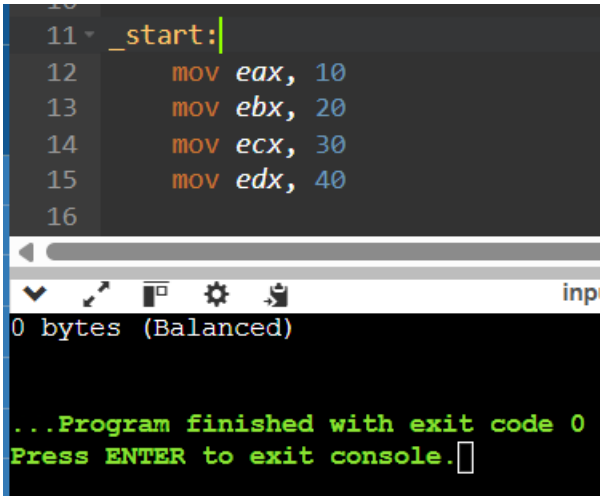

...Program finished with exit code 0
Press ENTER to exit console.
```

Problem 3: Multi-Register State Preservation & Arithmetic Subroutine

Description: Write a main program initializing EAX, EBX, ECX, EDX. Call a subroutine calculate_foo() that modifies these registers. The subroutine must PUSH all modified registers upon entry and POP them in reverse order before RET, ensuring zero net ESP offset.

Test Case	Initial Register State	Subroutine Operation	Final ESP Offset
Test Case 3.1	EAX=10, EBX=20, ECX=30, EDX=40	Multiply/Add with PUSH/POP	0 bytes (Balanced)
Test Case 3.2	EAX=5, EBX=5, ECX=10, EDX=2	Division/Accumulation	0 bytes (Balanced)
Test Case 3.3	EAX=100, EBX=50, ECX=25, EDX=5	Subtraction/Scaling	0 bytes (Balanced)
Test Case 3.4	EAX=1, EBX=2, ECX=3, EDX=4	Bitwise AND/OR operations	0 bytes (Balanced)
Test Case 3.5	EAX=88, EBX=11, ECX=4, EDX=2	Modulo & Division	0 bytes (Balanced)
Test Case 3.6	EAX=0, EBX=15, ECX=15, EDX=0	Zero-state arithmetic	0 bytes (Balanced)

Test Case 3.1



Test Case 3.2

```
10
11  _start:
12      mov eax, 5
13      mov ebx, 5
14      mov ecx, 10
15      mov edx, 2
16
```

```
0 bytes (Balanced)


...Program finished with exit code 0
Press ENTER to exit console.
```

Test Case 3.3

```
11  _start:
12      mov eax, 100
13      mov ebx, 50
14      mov ecx, 25
15      mov edx, 5
16
```

```
0 bytes (Balanced)


...Program finished with exit code 0
Press ENTER to exit console.
```

Test Case 3.4

```
11  _start:
12      mov eax, 1
13      mov ebx, 2
14      mov ecx, 3
15      mov edx, 4
16
```

```
0 bytes (Balanced)


...Program finished with exit code 0
Press ENTER to exit console.
```

Test Case 3.5

```
11  _start:
12      mov eax, 88
13      mov ebx, 11
14      mov ecx, 4
15      mov edx, 2
16
```

inp

```
) bytes (Balanced)


..Program finished with exit code 0
Press ENTER to exit console.
```

Test Case 3.6

```
11  _start:
12      mov eax, 0
13      mov ebx, 15
14      mov ecx, 15
15      mov edx, 0
16
```

input

```
0 bytes (Balanced)


...Program finished with exit code 0
Press ENTER to exit console.
```

Problem 4: Decimal to Binary Digit Conversion Using Stack Division

Description: Write an assembly program that takes an unsigned 16-bit integer, repeatedly divides it by 2 using DIV, and pushes the remainder bits onto the stack. After division completes, pop the bits iteratively to construct and display the binary string representation.

Test Case	Input Decimal Number	Expected Binary Output
Test Case 4.1	5	101
Test Case 4.2	10	1010
Test Case 4.3	15	1111
Test Case 4.4	8	1000
Test Case 4.5	25	11001
Test Case 4.6	42	101010

COMPILATION PROCESS/OUTPUT

Test Case 4.1:

```
@kizern →/workspaces/lab-exer-2 (main) $ nasm -f elf32 p4.asm -o p4.o
@kizern →/workspaces/lab-exer-2 (main) $ ld -m elf_i386 p4.o -o p4
@kizern →/workspaces/lab-exer-2 (main) $ ./p4
Enter decimal number (0-65535): 5
101
```

Test Case 4.2:

```
@kizern →/workspaces/lab-exer-2 (main) $ ./p4
Enter decimal number (0-65535): 10
1010
```

Test Case 4.3:

```
@kizern →/workspaces/lab-exer-2 (main) $ ./p4
Enter decimal number (0-65535): 15
1111
```

Test Case4.4:

```
@kizern →/workspaces/lab-exer-2 (main) $ ./p4
Enter decimal number (0-65535): 8
1000
```

Test Case 4.5:

```
@kizern →/workspaces/lab-exer-2 (main) $ ./p4
Enter decimal number (0-65535): 25
11001
```

Test Case 4.6:

```
@kizern ➜/workspaces/lab-exer-2 (main) $ ./p4
Enter decimal number (0-65535): 42
101010
```

Problem 5: Stack-Based Array Element Reversal & Summation

Description: Write an assembly program that pushes an array of 5 doubleword integers onto the stack. Then, pop them back into a new destination array in reverse order, while simultaneously computing their cumulative sum in EAX.

Test Case	Input Array Elements	Expected Reversed Array	Expected Sum (EAX)
Test Case 5.1	[1, 2, 3, 4, 5]	[5, 4, 3, 2, 1]	15
Test Case 5.2	[10, 20, 30, 40, 50]	[50, 40, 30, 20, 10]	150
Test Case 5.3	[2, 4, 6, 8, 10]	[10, 8, 6, 4, 2]	30
Test Case 5.4	[5, 5, 5, 5, 5]	[5, 5, 5, 5, 5]	25
Test Case 5.5	[0, 10, 20, 30, 40]	[40, 30, 20, 10, 0]	100
Test Case 5.6	[3, 1, 4, 1, 5]	[5, 1, 4, 1, 3]	14

COMPILATION PROCESS/OUTPUT

Test Case 5.1:

```
@kizern ➜/workspaces/lab-exer-2 (main) $ nasm -f elf32 p5.asm -o p5.o
@kizern ➜/workspaces/lab-exer-2 (main) $ ld -m elf_i386 p5.o -o p5
@kizern ➜/workspaces/lab-exer-2 (main) $ ./p5
Enter 5 integers separated by spaces: 1 2 3 4 5
Reversed array: 5 4 3 2 1
Sum: 15
```

Test Case 5.2:

```
@kizern ➜/workspaces/lab-exer-2 (main) $ ./p5
Enter 5 integers separated by spaces: 10 20 30 40 50
Reversed array: 50 40 30 20 10
Sum: 150
```

Test Case 5.3:

```
@kizern ➜/workspaces/lab-exer-2 (main) $ ./p5
Enter 5 integers separated by spaces: 2 4 6 8 10
Reversed array: 10 8 6 4 2
Sum: 30
```

Test Case 5.4:

```
@kizern →/workspaces/lab-exer-2 (main) $ ./p5
Enter 5 integers separated by spaces: 5 5 5 5 5
Reversed array: 5 5 5 5 5
Sum: 25
```

Test Case 5.5:

```
@kizern →/workspaces/lab-exer-2 (main) $ ./p5
Enter 5 integers separated by spaces: 0 10 20 30 40
Reversed array: 40 30 20 10 0
Sum: 100
```

Test Case 5.6:

```
@kizern →/workspaces/lab-exer-2 (main) $ ./p5
Enter 5 integers separated by spaces: 3 1 4 1 5
Reversed array: 5 1 4 1 3
Sum: 14
```

Part II: Open-Ended Conceptual & Critical Thinking Questions

Question 1: Stack Growth Direction

In x86 architecture, the stack grows downward toward lower memory addresses. Explain the architectural and performance reasons why the stack was designed to grow downward from high memory to low memory rather than growing upward like the data segment. How does this interact with heap allocation in memory management?

- The x86 instruction set defines downward stack growth, so pushing a value decreases the stack pointer before storing it. In a traditional memory layout, the stack and heap grow toward each other, allowing both to expand into available space. This arrangement improves memory flexibility, but downward growth itself is not inherently faster than upward growth.

Question 2: Byte vs. Word/Doubleword Push Limitations

Module notes state that 'only words or doublewords could be saved into the stack, not a byte.' Analyze why x86 hardware restricts stack operations to 16-bit words or 32-bit doublewords. What would happen to stack alignment and the Stack Pointer (ESP) if individual 8-bit registers or memory bytes were pushed directly without padding?

- In 32-bit x86, PUSH supports word and doubleword operands, matching common register and return-address sizes, but has no byte-sized stack operation. A hypothetical byte push would decrease ESP by one, potentially breaking alignment and making subsequent accesses less efficient or incompatible with calling conventions. A byte can instead be extended to a doubleword before pushing, preserving four-byte alignment when the stack is already aligned.

Question 3: Stack Frames and Recursive Overflows

When implementing recursive algorithms (such as the factorial problem in Module 5), each recursive call pushes return addresses and local register states onto the stack, creating stack frames. What occurs at the hardware and operating system level when recursion depth exceeds allocated stack segment boundaries (Stack Overflow)? How can programmers mitigate this risk?

- Each recursive call consumes stack space for its return address and any saved registers, arguments, and local variables. When an access exceeds the permitted stack region, hardware protection can trigger a fault that the operating system typically handles by terminating the program. Programmers can reduce

this risk by ensuring reachable base cases, limiting recursion depth, using iteration, or reducing local storage.

Question 4: Calling Conventions and Stack Cleanup

In low-level programming, responsibility for cleaning up parameters pushed onto the stack before a function call can fall either on the caller (e.g., cdecl) or the callee (e.g., stdcall). Compare these two approaches: what are the security and performance implications of each when dealing with variable argument functions (like printf)?

- In cdecl, the caller removes arguments from the stack which allows variable-argument functions such as prinf since the caller knows how much bytes it is given. With stdcall, the callee removes a fixed amount of argument data, which reduces repetitive code cleanuo but it doesn't guarantee faster compilation. Neither convention inherently prevents any vulnerability and mismatched cleanup responsibility can leave the stack pointer incorrect and cause crashes.